Complexité amortie

1. Introduction

1.1 Introduction

De manière générale, il était assez facile de calculer la complexité d'un algorithme.

Il suffisait de voir les boucles présentes dans l'algorithme, ainsi que le nombre de fois que chaque boucle était exécutée.

Sauf que parfois, cette analyse n'offre pas une vraie image de la complexité, en particulier quand la complexité d'une opération varie à chaque instant d'un algorithme.

L'idée à partir de maintenant, c'est de compter, pour chaque opération, le nombre total de fois exécutée durant l'algorithme.

C'est ce qu'on appelle une **analyse amortie** de la complexité.

1.2 Les concepts

Dans ce cours, on va introduire diverses méthodes de balayage :

- Méthode des 2 pointeurs
- Fenêtre glissante

2. Méthode des 2 pointeurs

2.1 Qu'est-ce que c'est ?

C'est une méthode qui consiste à itérer dans une liste, à l'aide de 2 pointeurs l (*left*) et r (*right*).

Ces 2 pointeurs se déplacent **seulement** dans 1 seul sens, c'est ce qui garantit que l'algorithme est efficace.

L'idée avec cette méthode, c'est qu'on peut s'amuser à maintenir dans une liste A, une sous-liste $A[l, r]$ qui vérifie une condition particulière.

2.2 Sous-liste de somme x

On nous fournit une liste A d'entiers **positifs**, et une valeur x.

Énoncé : Existe-t-il une sous-liste qui a pour somme x ?

Par exemple avec la liste ci-dessous, et $x = 8$,

1	3	2	5	1	1	2	3

La sous-liste colorée suivante est une solution :

1	3	2	5	1	1	2	3

2.3 Solution

L'idée est la suivante, on va maintenir une variable y qui sera la somme totale de la fenêtre considérée de l inclus à r exclus.

Nos 2 pointeurs l et r sont au début de la liste.

1. On avance r d'une case vers la droite :

$$y = 1$$

1	3	2	5	1	1	2	3
l	r						

y est inférieur à x, on continue alors.

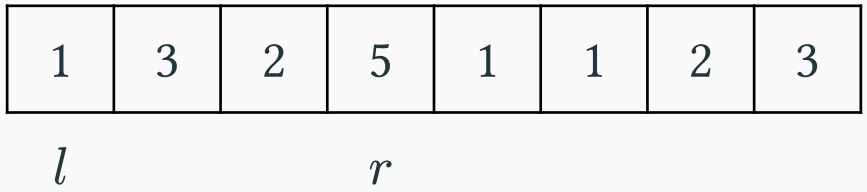
$$y = 4$$

1	3	2	5	1	1	2	3

l r

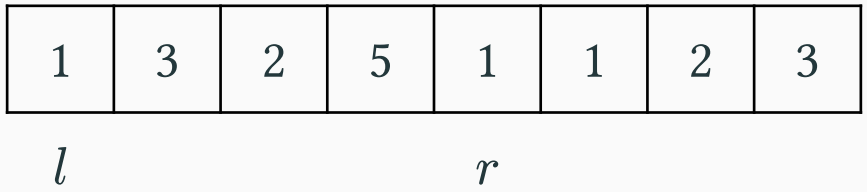
y est inférieur à x, on continue alors.

$y = 6$



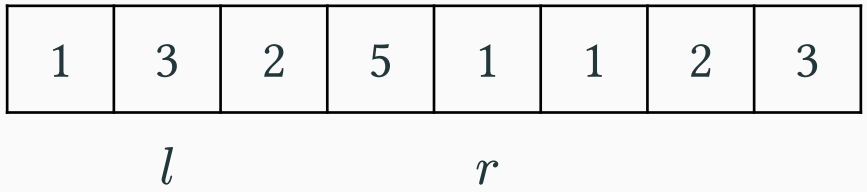
y est inférieur à x, on continue alors.

$$y = 11$$



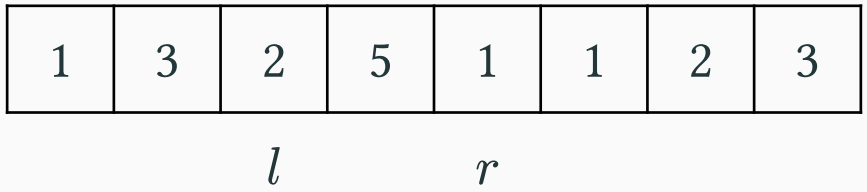
y est supérieur à x, on avance maintenant l pour réduire y.

$$y = 10$$



y est supérieur à x, on avance encore l pour réduire y.

$$y = 7$$



y est inférieur à x, on peut recommencer à avancer r.

$$y = 8$$



$y = x$, nous avons trouvé une sous-liste de somme x.

Dans le cas où une sous-liste n'est pas trouvée, r atteindra la fin de la liste, et dans ce cas, une solution n'existe pas.

3. Fenêtre glissante

L'idée est similaire à la méthode précédente, avec un petit plus, la taille de la sous-liste est fixée.

Ici on parle de **fenêtre** car la taille de la sous-liste est fixée durant le balayage.

Exemple :

On nous fournit une liste A d'entiers, et une valeur k.

Énoncé : Donner dans l'ordre, la somme de chaque fenêtre de taille k dans A.

3.2 Solution

Au lieu de recalculer la somme pour chaque fenêtre ($\mathcal{O}(n^2)$), on fonctionne par contribution.

Dans la solution, on suppose que k est égal à 3.

On somme d'abord les éléments en gris, c'est notre fenêtre glissante au début.

$$y = 6$$



L'idée en déplaçant la fenêtre d'un pas vers la droite, et d'ajouter l'élément suivant :

$$y = 11$$



Et de retirer la première valeur de la fenêtre :

$$y = 10$$



En faisant cela, on garantit que chaque élément est traité au plus 2 fois, ce qui donne une complexité $\mathcal{O}(n)$.

4. Crédits

Auteurs : Bilel MAHDJOUB

Pour le **Bureau des technologies** de **Polytech Nice Sophia**

5. Sources et ressources

- CPH book - Amortized analysis